Group 44
Roll No - 2103109

Thadomal Shahani Engineering College

Bandra (W.), Mumbai- 400 050.

CERTIFICATE

Certify that Mr./Miss Arnav Malvia

of Computer Engineering Department, Semester VIII with

Roll No. 2103109 has completed a course of the necessary experiments in the subject Distributed Computing under my supervision in the **Thadomal Shahani Engineering College** Laboratory in the year 2024 - 2025

Teacher In-Charge

Head of the Department

Date 09/4/2025

Principal

THADOMAL SHAHANI
TSEC
ENGINEERING COLLEGE

CONTENTS

EXPERIMENT 1

AIM: Write a program to implement inter process communication

Theory:

In contemporary computing environments, multiple processes often execute simultaneously to improve system efficiency, responsiveness, and throughput. These processes, while independent, may need to coordinate actions or exchange data to perform a common task. This requirement leads to the concept of **Inter-Process Communication (IPC)**.

Inter-process communication refers to the techniques and mechanisms that allow processes to communicate and share data with each other. Processes may be part of the same program or completely separate programs. Since each process typically runs in its own isolated memory space, IPC provides the necessary bridge to enable this data exchange.

IPC is a fundamental concept in operating systems, distributed systems, client-server models, and multitasking applications. It enables coordination among processes in real-time, improves system modularity, and supports the development of scalable and robust software systems. IPC is also crucial in systems where concurrent execution of tasks is essential, such as embedded systems, transaction processing systems, and web servers.

The main challenge in IPC is to ensure that data is transferred securely, efficiently, and without conflicts. Different IPC mechanisms are designed to address these challenges in different scenarios. Some are simple to implement and ideal for unidirectional communication (like pipes), while others are more sophisticated and suitable for bidirectional or asynchronous communication (like message queues and shared memory with semaphores).

Key Concepts:

- **Process**: An executing instance of a program, with its own memory and resources.
- **Communication**: The exchange of information or data between two entities—in this case, processes.
- **Synchronization**: Mechanism to ensure coordinated access to shared resources, often using locks, semaphores, or signals.
- **Race Condition**: A condition where two or more processes access shared data and try to change it at the same time, leading to unpredictable results.
- **Deadlock**: A state where two or more processes are waiting indefinitely for resources held by each other.

Applications of IPC:

- Communication between different modules of a distributed software system.
- Real-time interaction between sensor processes and controller processes in embedded systems.

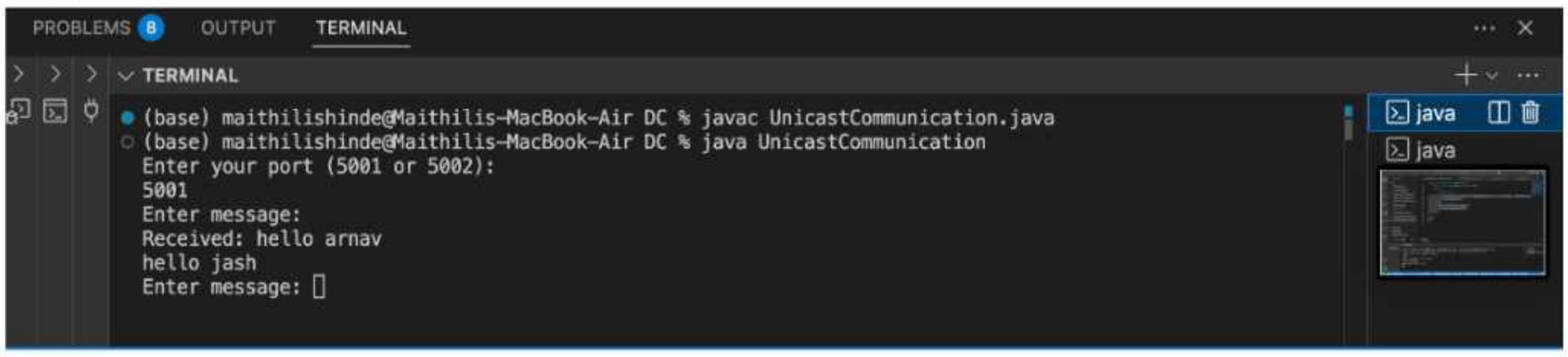
- Enabling background services and daemons to interact with user interfaces.
- Multiprocessing environments in operating systems to improve performance.
- Chat applications, multiplayer games, and client-server networks.

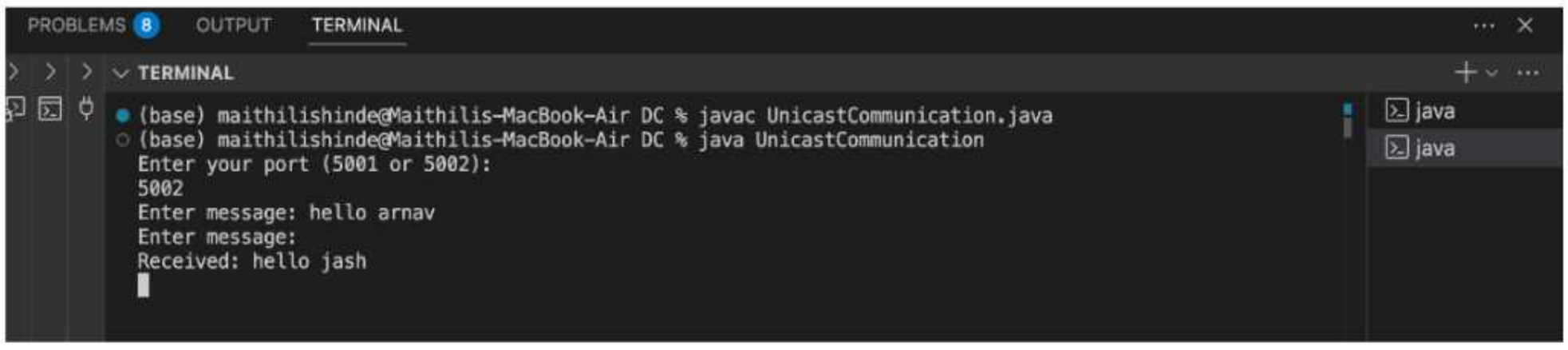
Code & Output:

```
import java.net.*;
import java.io.*;

public class UnicastCommunication {
    private static final int PORT1 = 5001; // Port for Process 1
    private static final int PORT2 = 5002; // Port for Process 2
    private static final String LOCALHOST = "127.0.0.1";

    public static void main(String[] args) {
        try {
            BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));

            System.out.println("Enter your port (5001 or 5002): ");
            int myPort = Integer.parseInt(reader.readLine());
            int peerPort = (myPort == PORT1) ? PORT2 : PORT1; // Decide the peer's port

            DatagramSocket socket = new DatagramSocket(myPort);

            // Start receiving messages in a separate thread
            new Thread(() -> {
                try {
                    byte[] receiveBuffer = new byte[1024];
                    while (true) {
                        DatagramPacket packet = new DatagramPacket(receiveBuffer,
receiveBuffer.length);
                        socket.receive(packet);
                        String message = new String(packet.getData(), 0, packet.getLength());
                        System.out.println("\nReceived: " + message);
                    }
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }).start();

            // Sending messages
            while (true) {
                System.out.print("Enter message: ");
                String message = reader.readLine();
                if (message.equalsIgnoreCase("exit")) break;
```

```
            byte[] sendBuffer = message.getBytes();
            DatagramPacket sendPacket = new DatagramPacket(
                  sendBuffer, sendBuffer.length, InetAddress.getByName(LOCALHOST),
peerPort);
            socket.send(sendPacket);
         }

         socket.close();
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
}
```

Output:





EXPERIMENT 2

AIM: Write a program to implement Group communication

Theory:

Group communication is a fundamental concept in distributed systems where processes (or nodes) cooperate by forming groups. The key idea is that instead of sending a message to individual processes, a process can send a single message to a group, and all the members of that group receive the message. This model simplifies the development of distributed applications by abstracting the complexity of individual communications and ensuring consistency in message delivery.

In real-world applications, group communication is essential for systems like collaborative platforms, multiplayer games, distributed databases, sensor networks, and fault-tolerant systems. For example, in a chatroom application, when one user sends a message, it should be delivered to all participants in the group simultaneously.

Group communication ensures that data sent to a group is received consistently by all group members. Advanced group communication systems may support properties like reliable delivery, ordered delivery, and atomic delivery, which are critical for maintaining consistency in replicated systems or databases.

Group communication can be implemented using multicast sockets, broadcasting over a LAN, or by maintaining a list of group members and sending the message individually to each of them (simulated multicast). The design choice depends on the network infrastructure and application requirements.

Key Concepts:

- **Multicast**: The process of sending a message to a group of interested recipients rather than just one.
- **Group**: A collection of processes or nodes that are intended to communicate together.
- **Reliability**: Ensuring all group members receive the message without loss.
- **Ordering**: Ensuring all messages are received in the same order by all members.
- **Membership Management**: Keeping track of which processes are in the group at any time.

Types of Group Communication:

1. **Unreliable Group Communication**
    - No guarantee that all members receive the message.
    - Simple and fast but not suitable for critical applications.

2. **Reliable Group Communication**

- Guarantees message delivery to all group members.
- May involve acknowledgments or retransmissions.

3. **Causal and Total Order Group Communication**
   - Ensures all members receive messages in the same causal or total order.
   - Important for consistency in distributed databases.

Applications:

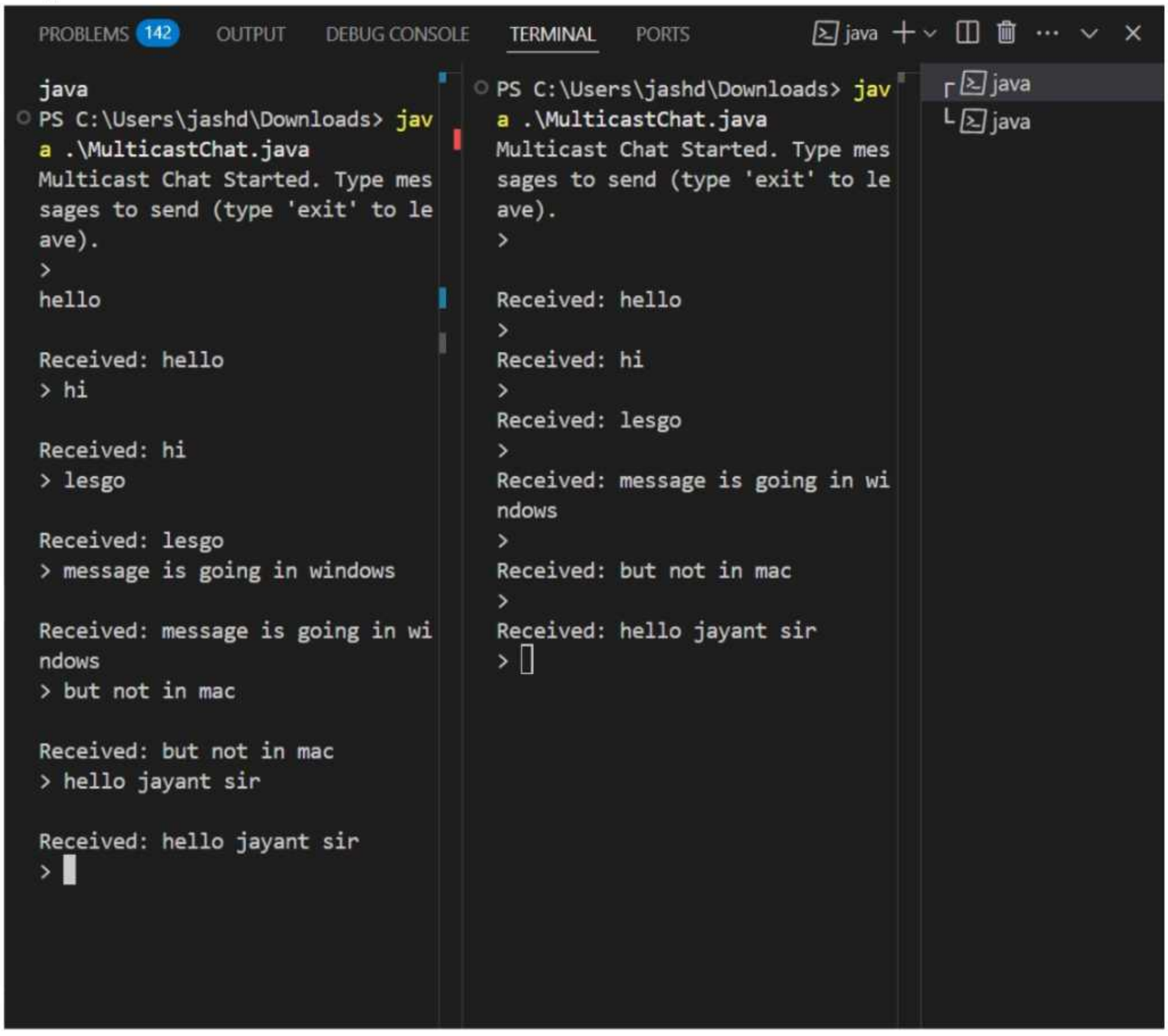
- Real-time collaborative tools (e.g., Google Docs)
- Multiplayer online games
- Distributed file systems
- Chat applications (group messaging)
- Cluster management in distributed computing systems

Code & Output:

```
import java.io.*;
import java.net.*;

public class MulticastChat {
    private static final String MULTICAST_GROUP = "230.0.0.1";
    private static final int PORT = 5000;
    private MulticastSocket socket;
    private InetAddress group;
    private NetworkInterface networkInterface;

    public MulticastChat() throws IOException {
        socket = new MulticastSocket(PORT);
        group = InetAddress.getByName(MULTICAST_GROUP);
        networkInterface = NetworkInterface.getByInetAddress(InetAddress.getLocalHost());

        if (networkInterface == null) {
            throw new IOException("No suitable network interface found.");
        }

        socket.joinGroup(new InetSocketAddress(group, PORT), networkInterface);
    }

    public void startListening() {
        new Thread(() -> {
            try {
                byte[] buffer = new byte[1024];
                while (true) {
                    DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
```

```
                socket.receive(packet);
                System.out.println("\nReceived: " + new String(packet.getData(), 0,
packet.getLength()));
                System.out.print("> ");
            }
        } catch (IOException e) {
            System.out.println("Exited multicast group.");
        }
    }).start();
}

public void sendMessage(String message) throws IOException {
    byte[] buffer = message.getBytes();
    socket.send(new DatagramPacket(buffer, buffer.length, group, PORT));
}

public void leaveGroup() throws IOException {
    socket.leaveGroup(new InetSocketAddress(group, PORT), networkInterface);
    socket.close();
}

public static void main(String[] args) {
    try {
        MulticastChat chat = new MulticastChat();
        chat.startListening();
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Multicast Chat Started. Type messages to send (type 'exit' to
leave).\n> ");

        String message;
        while (!(message = reader.readLine()).equalsIgnoreCase("exit")) {
            chat.sendMessage(message);
        }
        chat.leaveGroup();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}
```

Output:


PROBLEMS 142
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
java
java
PS C:\Users\jashd\Downloads> java .\MulticastChat.java
Multicast Chat Started. Type messages to send (type 'exit' to leave).
>
hello
Received: hello
> hi
Received: hi
> lesgo
Received: lesgo
> message is going in windows
Received: message is going in windows
> but not in mac
Received: but not in mac
> hello jayant sir
Received: hello jayant sir
>
PS C:\Users\jashd\Downloads> java .\MulticastChat.java
Multicast Chat Started. Type messages to send (type 'exit' to leave).
>
Received: hello
>
Received: hi
>
Received: lesgo
>
Received: message is going in windows
>
Received: but not in mac
>
Received: hello jayant sir
>
java
java

EXPERIMENT 3

AIM: Write a program to simulate Clock synchronisation

Theory:

In distributed systems, each node or process may run on a separate physical machine with its own local clock. Due to differences in hardware and drift rates, these clocks may not be synchronized, leading to inconsistencies in time-dependent operations like logging, ordering of events, and coordination tasks. Clock synchronization ensures that all the nodes in a distributed system agree on a common notion of time, which is essential for system correctness and reliability.

Since there's no global clock in distributed systems, synchronization must be achieved through communication between nodes. Various algorithms have been developed to minimize the time offset and achieve consistent timekeeping across all nodes. These algorithms typically involve periodic exchange of timestamped messages, calculating clock drifts, and adjusting local clocks accordingly.

Key Concepts:

- **Clock Drift**: The gradual divergence of a clock's time from the real time or another clock's time.
- **Synchronization**: The process of adjusting clocks so that they agree within a certain precision.
- **Round Trip Time (RTT)**: The time taken for a message to travel from source to destination and back.
- **Offset**: The amount by which a clock differs from a reference clock.
- **Skew**: The difference in rate at which two clocks progress over time.

Popular Clock Synchronization Algorithms:

1. **Cristian's Algorithm:**
    - A process queries a time server.
    - Estimates delay using round-trip time (RTT).
    - Adjusts its clock to the server's time plus half the RTT.

2. **Berkeley's Algorithm:**
    - A master node polls all slaves for their local time.
    - Computes an average time (excluding faulty or delayed responses).
    - Sends adjustment values (not absolute time) to each process.

3. **Network Time Protocol (NTP):**
    - Real-world protocol with a hierarchy of time servers.
    - Uses multiple sources to improve accuracy and fault tolerance.

- Ensures synchronization within milliseconds over the internet.

4. **Lamport's Logical Clock:**
   - Focuses on event ordering rather than actual time.
   - Each process maintains a counter; messages carry timestamps.
   - Helps ensure causality in distributed systems (i.e., if event A → event B, then timestamp(A) < timestamp(B)).

Applications:

- Logging and auditing in distributed systems
- Event ordering (e.g., in databases or message queues)
- Time-sensitive applications like stock trading platforms
- Real-time collaborative applications
- Synchronizing file systems and backups

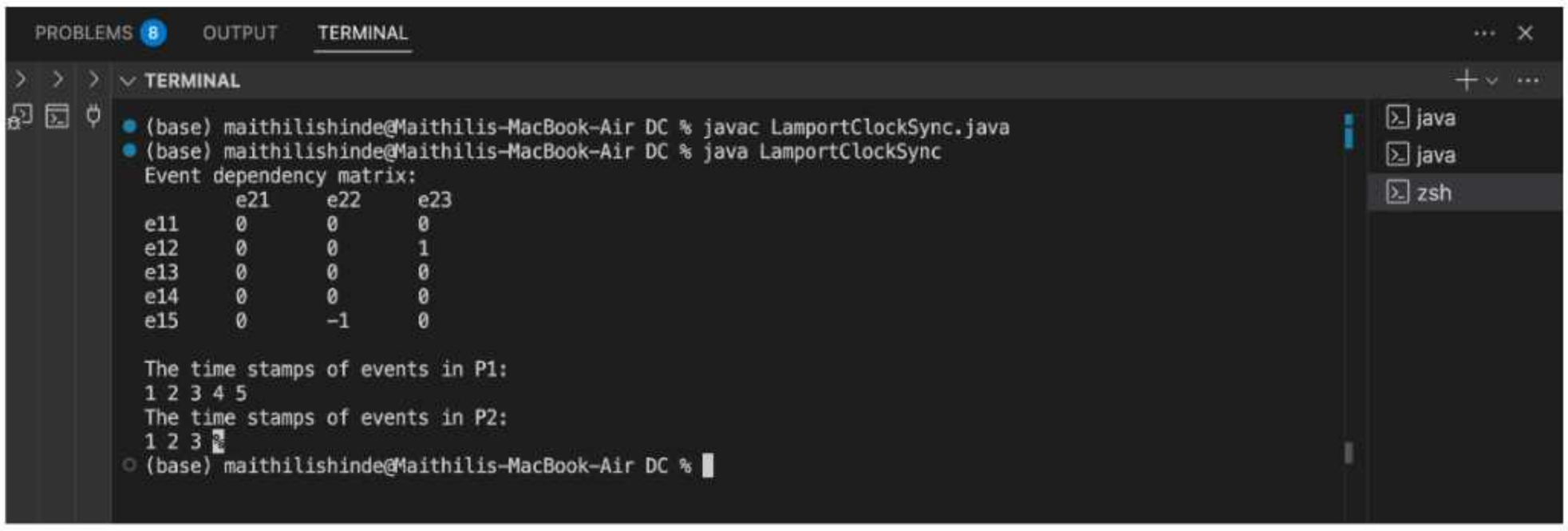
Code & Output:

```
import java.util.*;

public class LamportClockSync {

    // Function to find the maximum of two numbers
    static int max(int a, int b) {
        return (a > b) ? a : b;
    }

    // Function to display logical timestamps
    static void display(int e1, int e2, int[] p1, int[] p2) {
        System.out.println("\nThe time stamps of events in P1:");
        for (int i = 0; i < e1; i++) {
            System.out.print(p1[i] + " ");
        }

        System.out.println("\nThe time stamps of events in P2:");
        for (int i = 0; i < e2; i++) {
            System.out.print(p2[i] + " ");
        }
    }

    // Function to compute timestamps of events
    static void lamportLogicalClock(int e1, int e2, int[][] m) {
        int[] p1 = new int[e1];
        int[] p2 = new int[e2];

        // Initialize timestamps
```

```
for (int i = 0; i < e1; i++) {
    p1[i] = i + 1;
}
for (int i = 0; i < e2; i++) {
    p2[i] = i + 1;
}

// Display event matrix
System.out.println("Event dependency matrix:");
System.out.print("\t");
for (int i = 0; i < e2; i++) {
    System.out.print("e2" + (i + 1) + "\t");
}
System.out.println();

for (int i = 0; i < e1; i++) {
    System.out.print("e1" + (i + 1) + "\t");
    for (int j = 0; j < e2; j++) {
        System.out.print(m[i][j] + "\t");
    }
    System.out.println();
}

// Compute logical timestamps
for (int i = 0; i < e1; i++) {
    for (int j = 0; j < e2; j++) {
        if (m[i][j] == 1) { // Message sent
            p2[j] = max(p2[j], p1[i] + 1);
            for (int k = j + 1; k < e2; k++) {
                p2[k] = p2[k - 1] + 1;
            }
        }

        if (m[i][j] == -1) { // Message received
            p1[i] = max(p1[i], p2[j] + 1);
            for (int k = i + 1; k < e1; k++) {
                p1[k] = p1[k - 1] + 1;
            }
        }
    }

    // Display final timestamps
    display(e1, e2, p1, p2);
}

public static void main(String[] args) {
    int e1 = 5, e2 = 3;
```

```
int[][] m = new int[5][3];

// Message dependency matrix
m[0][0] = 0;
m[0][1] = 0;
m[0][2] = 0;
m[1][0] = 0;
m[1][1] = 0;
m[1][2] = 1;
m[2][0] = 0;
m[2][1] = 0;
m[2][2] = 0;
m[3][0] = 0;
m[3][1] = 0;
m[3][2] = 0;
m[4][0] = 0;
m[4][1] = -1;
m[4][2] = 0;

// Execute algorithm
lamportLogicalClock(e1, e2, m);
    }
}
```

Output :



EXPERIMENT 4

AIM: Write a program to simulate Election algorithm (Bully Algorithm)

Theory:

In a distributed system, some tasks may require a coordinator node (leader) responsible for managing shared resources or decision-making. However, the coordinator may fail or leave the network unexpectedly. In such cases, the system must elect a **new coordinator** dynamically — this is achieved using an **election algorithm**.

The **Bully Algorithm**, proposed by Garcia-Molina in 1982, is one of the most well-known election algorithms. It is used when all nodes in the system have **unique IDs**, and the node with the **highest ID** becomes the coordinator.

Working of the Bully Algorithm:

1. **Detection of Coordinator Failure:**
    - If a process notices that the current coordinator is not responding, it initiates an election.
2. **Election Process:**
    - The initiating process sends an **ELECTION** message to all processes with **higher IDs**.
    - If no process responds, it becomes the new coordinator.
    - If one or more higher-ID processes respond, they take over the election process.
3. **Coordinator Declaration:**
    - The process with the **highest ID**, after getting no response from any higher process, declares itself the coordinator.
    - It sends a **COORDINATOR** message to all processes.
4. **All processes update** their state to acknowledge the new coordinator.

Key Features:

- Assumes **synchronous communication** and **non-faulty links**.
- Higher-ID processes **"bully"** lower-ID ones out of the coordinator role.
- Works well when process IDs are static and known to all.

Advantages:

- Simple and easy to implement.
- Guarantees election of the highest-priority (highest-ID) process.
  Robust against failure of multiple lower-ID processes.

Limitations:

- Can generate a large number of messages in the case of multiple simultaneous elections.
- Assumes all processes know the IDs of others in the system.
- Assumes reliable message delivery.

Applications:

- Distributed database systems
- Leader election in replicated services
- Clustered systems or distributed consensus algorithms
- Master node election in systems like Hadoop or ZooKeeper

Code & Output:

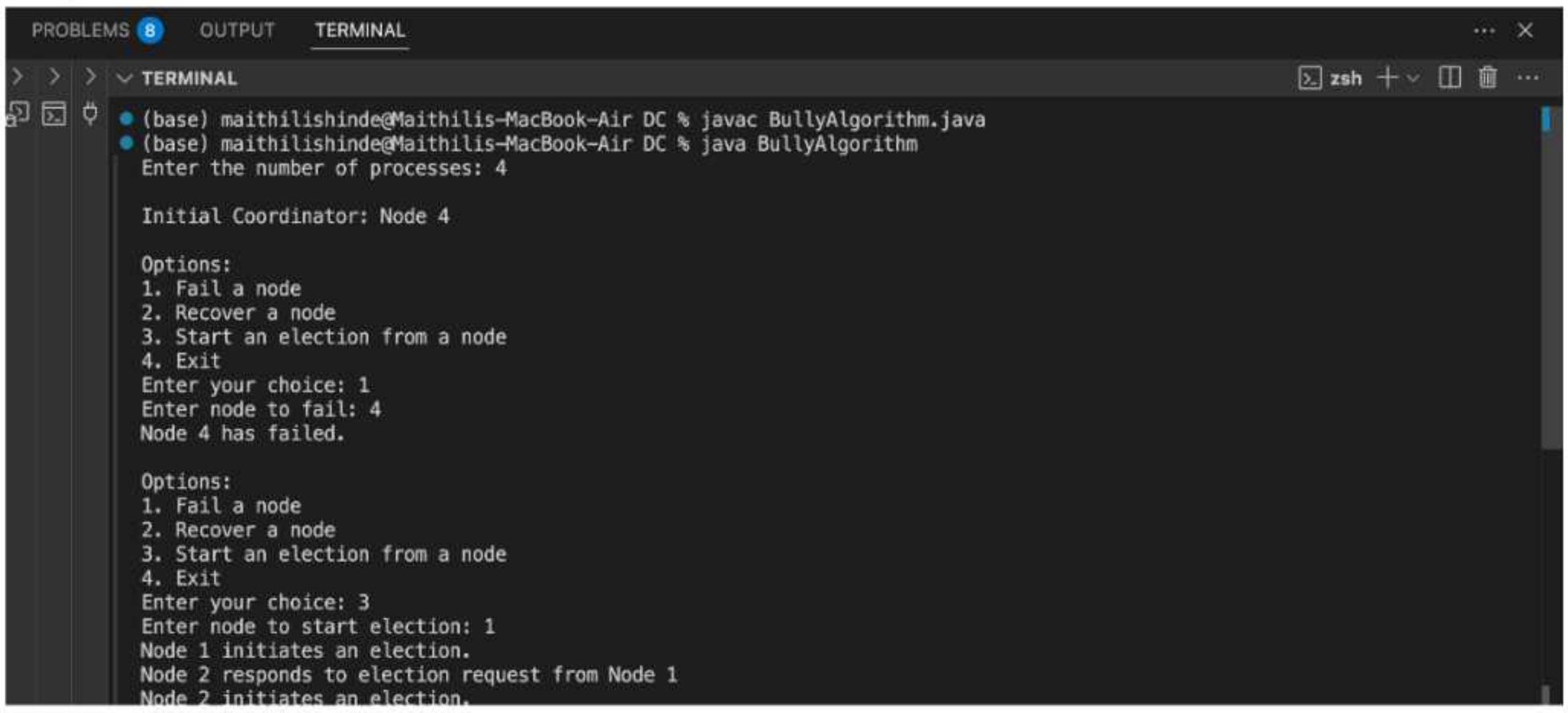
```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

// Class representing a node in the distributed system
class Node {
    private int nodeId;
    private boolean isCoordinator;
    private boolean isAlive;
    private List<Node> nodes;

    public Node(int nodeId, List<Node> nodes) {
        this.nodeId = nodeId;
        this.isCoordinator = false;
        this.isAlive = true;
        this.nodes = nodes;
    }

    public int getNodeId() {
        return nodeId;
    }

    public boolean isCoordinator() {
        return isCoordinator;
    }

    public boolean isAlive() {
        return isAlive;
    }
```

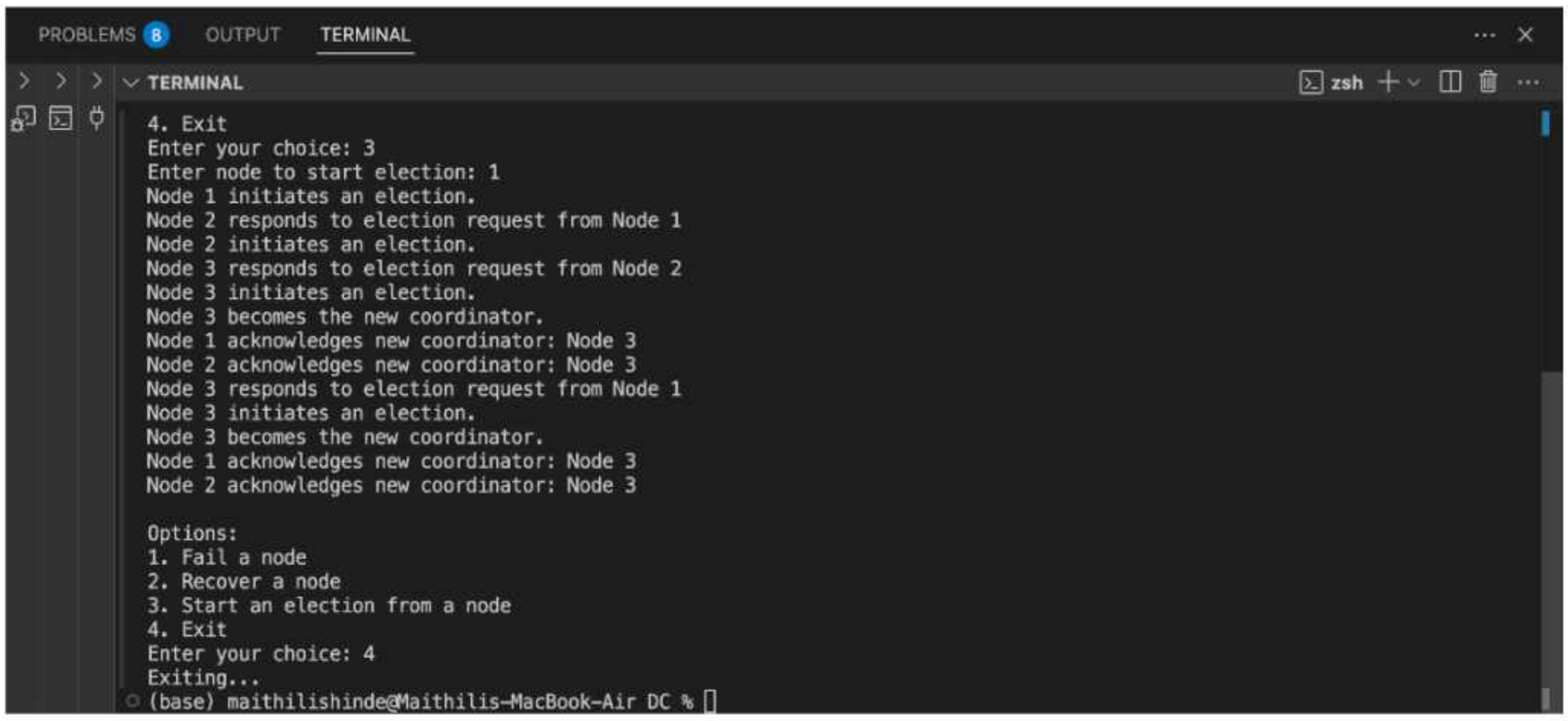
```
public void setCoordinator(boolean coordinator) {
    isCoordinator = coordinator;
}

public void failNode() {
    isAlive = false;
    isCoordinator = false;
    System.out.println("Node " + nodeId + " has failed.");
}

public void recoverNode() {
    isAlive = true;
    System.out.println("Node " + nodeId + " has recovered.");
}

// Initiate an election
public void initiateElection() {
    if (!isAlive) {
        System.out.println("Node " + nodeId + " is down and cannot initiate an election.");
        return;
    }

    System.out.println("Node " + nodeId + " initiates an election.");

        boolean higherNodeExists = false;

        for (Node node : nodes) {
            if (node.getNodeId() > this.nodeId && node.isAlive()) {
                higherNodeExists = true;
                System.out.println("Node " + node.getNodeId() + " responds to election request
from Node " + nodeId);
                node.initiateElection(); // Higher node starts election
            }
        }

        if (!higherNodeExists) {
            becomeCoordinator();
        }
}

// Become the coordinator
public void becomeCoordinator() {
    System.out.println("Node " + nodeId + " becomes the new coordinator.");
    this.isCoordinator = true;
    announceCoordinator();
}
```

```
// Announce the new coordinator to all nodes
public void announceCoordinator() {
    for (Node node : nodes) {
        if (node.getNodeId() != this.nodeId && node.isAlive()) {
            node.receiveCoordinatorMessage(this);
        }
    }
}

// Receive notification of new coordinator
    public void receiveCoordinatorMessage(Node newCoordinator) {
        System.out.println("Node " + nodeId + " acknowledges new coordinator: Node " +
newCoordinator.getNodeId());
    }
}

public class BullyAlgorithm {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Get number of processes
        System.out.print("Enter the number of processes: ");
        int numProcesses = scanner.nextInt();

// Create a list of nodes
List<Node> nodes = new ArrayList<>();
for (int i = 1; i <= numProcesses; i++) {
    nodes.add(new Node(i, nodes));
}

// Initial Coordinator is the highest node
nodes.get(numProcesses - 1).setCoordinator(true);
System.out.println("\nInitial Coordinator: Node " + numProcesses);

while (true) {
    System.out.println("\nOptions:");
    System.out.println("1. Fail a node");
    System.out.println("2. Recover a node");
    System.out.println("3. Start an election from a node");
    System.out.println("4. Exit");
    System.out.print("Enter your choice: ");

    int choice = scanner.nextInt();

    switch (choice) {
        case 1:
            System.out.print("Enter node to fail: ");
            int failNodeId = scanner.nextInt();
```

```
            if (failNodeId > 0 && failNodeId <= numProcesses) {
                nodes.get(failNodeId - 1).failNode();
            } else {
                System.out.println("Invalid node ID.");
            }
            break;

        case 2:
            System.out.print("Enter node to recover: ");
            int recoverNodeId = scanner.nextInt();
            if (recoverNodeId > 0 && recoverNodeId <= numProcesses) {
                nodes.get(recoverNodeId - 1).recoverNode();
            } else {
                System.out.println("Invalid node ID.");
            }
            break;

        case 3:
            System.out.print("Enter node to start election: ");
            int electionNodeId = scanner.nextInt();
            if (electionNodeId > 0 && electionNodeId <= numProcesses) {
                nodes.get(electionNodeId - 1).initiateElection();
            } else {
                System.out.println("Invalid node ID.");
            }
            break;

        case 4:
            System.out.println("Exiting...");
            scanner.close();
            return;

        default:
            System.out.println("Invalid choice. Try again.");
        }
    }
}
```

Output:


PROBLEMS 8
OUTPUT
TERMINAL
TERMINAL
zsh
(base) maithilishinde@Maithilis-MacBook-Air DC % javac BullyAlgorithm.java
(base) maithilishinde@Maithilis-MacBook-Air DC % java BullyAlgorithm
Enter the number of processes: 4
Initial Coordinator: Node 4
Options:
1. Fail a node
2. Recover a node
3. Start an election from a node
4. Exit
Enter your choice: 1
Enter node to fail: 4
Node 4 has failed.
Options:
1. Fail a node
2. Recover a node
3. Start an election from a node
4. Exit
Enter your choice: 3
Enter node to start election: 1
Node 1 initiates an election.
Node 2 responds to election request from Node 1
Node 2 initiates an election.


PROBLEMS 8
OUTPUT
TERMINAL
TERMINAL
zsh
4. Exit
Enter your choice: 3
Enter node to start election: 1
Node 1 initiates an election.
Node 2 responds to election request from Node 1
Node 2 initiates an election.
Node 3 responds to election request from Node 2
Node 3 initiates an election.
Node 3 becomes the new coordinator.
Node 1 acknowledges new coordinator: Node 3
Node 2 acknowledges new coordinator: Node 3
Node 3 responds to election request from Node 1
Node 3 initiates an election.
Node 3 becomes the new coordinator.
Node 1 acknowledges new coordinator: Node 3
Node 2 acknowledges new coordinator: Node 3
Options:
1. Fail a node
2. Recover a node
3. Start an election from a node
4. Exit
Enter your choice: 4
Exiting...
(base) maithilishinde@Maithilis-MacBook-Air DC %

EXPERIMENT 5

AIM: Write a program to simulate Mutual Exclusion (Raymond's Tree-Based Algorithm)

Theory:

In distributed systems, multiple processes may attempt to access shared resources simultaneously. To maintain data consistency and system integrity, **mutual exclusion** ensures that only **one process accesses a critical section (CS)** at a time.

While centralized and token-based algorithms exist, they can be inefficient or prone to bottlenecks. **Raymond's Tree-Based Algorithm** improves scalability and reduces message overhead by organizing processes in a **logical tree** structure, where each node knows only its parent and children.

This algorithm is especially suited for distributed environments where processes are loosely connected and direct communication with every other process is impractical.

Key Concepts:

- The system is arranged as a **logical directed tree**.
- A **token** is used to grant access to the critical section (CS).
- Only one copy of the token exists.
- Each process maintains a **request queue** and a **holder** pointer indicating from where it expects the token.
- Requests propagate up the tree towards the token holder.

Working Principle:

1. **Requesting the Token:**
    - If a process wants to enter the CS and doesn't have the token, it sends a request to its parent (the node indicated by the `holder`).
    - This request is forwarded recursively up the tree until it reaches the token holder.

2. **Token Handling:**
    - If the token holder is idle, it sends the token back down the path to the requester.
    - If it is using the token, it queues incoming requests and sends the token once it exits the CS.

3. **Token Transfer:**
    - When a node with the token exits the CS, it checks its queue.

- If the queue is not empty, it sends the token to the first requestor and updates the `holder`.

4. **Fairness and Efficiency:**
   - Requests are served in **FIFO** order.
   - Message complexity is **O(log N)** in a balanced tree, which is more efficient than some other algorithms.

Advantages:

- Low message overhead compared to Ricart-Agrawala.
  Efficient in large-scale systems with hierarchical topologies.
  Reduces unnecessary broadcasting.

Limitations:

- Failure of any node in the path may halt progress.
  Logical tree structure must be maintained and updated.
- Slightly more complex to implement due to tree management.

Applications:

- Distributed databases
- Resource allocation in distributed file systems
- Lock management in peer-to-peer and hierarchical systems
- Grid and cloud computing infrastructures

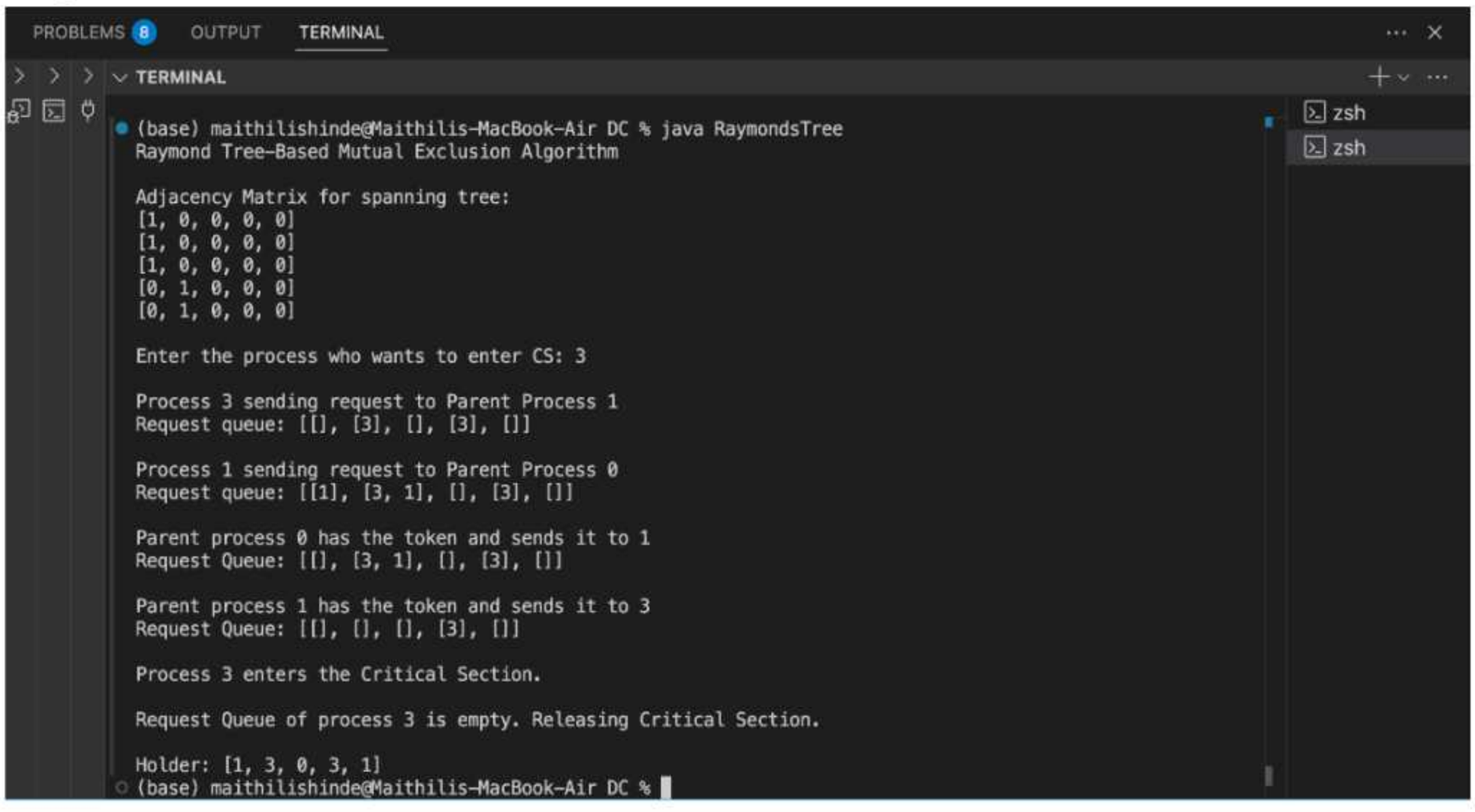
Code & Output:

```
import java.util.*;

public class RaymondsTree {
    static int n = 5;
    @SuppressWarnings("unchecked")
    static List<Integer>[] requestQueue = (List<Integer>[]) new ArrayList[n];
    static int[] holder = {0, 0, 0, 1, 1};  // Holder array
    static int[] token = {1, 0, 0, 0, 0};  // Token status
    static int[][] adjMatrix = {
        {1, 0, 0, 0, 0},
        {1, 0, 0, 0, 0},
        {1, 0, 0, 0, 0},
        {0, 1, 0, 0, 0},
        {0, 1, 0, 0, 0}
```

```
};

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // Initialize request queues
    for (int i = 0; i < n; i++) {
        requestQueue[i] = new ArrayList<>();
    }

System.out.println("Raymond Tree-Based Mutual Exclusion Algorithm\n");
System.out.println("Adjacency Matrix for spanning tree:");
for (int[] row : adjMatrix) {
    System.out.println(Arrays.toString(row));
}

// Read the process that wants to enter CS
System.out.print("\nEnter the process who wants to enter CS: ");
int reqProcess = scanner.nextInt();

int parent = findParent(reqProcess);
while (token[reqProcess] != 1) {
    int child = requestQueue[parent].remove(0);  // Get the first request in queue
    holder[parent] = child;
    holder[child] = 0;
    token[parent] = 0;
    token[child] = 1;

        requestQueue[parent].clear();

    System.out.println("\nParent process " + parent + " has the token and sends it to " +
child);
    System.out.println("Request Queue: " + Arrays.deepToString(requestQueue));

    parent = child;
  }

  if (token[parent] == 1 && !requestQueue[parent].isEmpty() &&
requestQueue[parent].get(0) == parent) {
    requestQueue[parent].remove(0);
    holder[parent] = parent;
    System.out.println("\nProcess " + parent + " enters the Critical Section.");
  }

  if (requestQueue[parent].isEmpty()) {
    System.out.println("\nRequest Queue of process " + parent + " is empty. Releasing
Critical Section.");
  }
```

```
    System.out.println("\nHolder: " + Arrays.toString(holder));
    scanner.close();
}

private static int findParent(int reqProcess) {
    requestQueue[reqProcess].add(reqProcess);
    int parent = -1;

    for (int i = 0; i < n; i++) {
        if (adjMatrix[reqProcess][i] == 1) {
            parent = i;
            requestQueue[parent].add(reqProcess);
            break;
        }
    }

    System.out.println("\nProcess " + reqProcess + " sending request to Parent Process " +
parent);
    System.out.println("Request queue: " + Arrays.deepToString(requestQueue));

    if (token[parent] == 1) {
        return parent;
    } else {
        return findParent(parent);
    }
  }
}
```

Output:



EXPERIMENT 6

AIM: Write a program to simulate Deadlock management in Distributed systems (Banker's Algorithm)

Theory:

In distributed systems, processes compete for shared resources such as memory, CPU cycles, and data files. This competition can lead to **deadlocks**, where processes are stuck waiting indefinitely for resources held by others.

To prevent such situations, **Dijkstra** proposed the **Banker's Algorithm**, which is a **deadlock avoidance** strategy. The algorithm checks if granting a resource request leads the system into an unsafe state. If so, the request is denied until it becomes safe to proceed.

Named after a banker allocating funds to clients while ensuring that all of them can eventually be satisfied, this algorithm is widely used for resource allocation in systems that require high reliability and safety.

Key Concepts:

- **Safe State**: A state where there exists at least one sequence in which all processes can complete without leading to a deadlock.
- **Unsafe State**: A state that may lead to a deadlock if further resource allocations are made.
- **Need Matrix**: Represents the remaining resources each process may still request.
- **Available Vector**: Represents the number of available instances of each resource type.
- **Allocation Matrix**: Tracks how many resources are currently allocated to each process.
- **Maximum Matrix**: Tracks the maximum demand of each process.

Working of the Banker's Algorithm:

1. **Resource Request Handling**:
    - When a process requests resources, the system checks:
        - If the request does not exceed the process's declared maximum.
        - If the resources are available.

2. **Safety Check**:
    - Temporarily allocate the requested resources.
    - Perform a **safety algorithm** to determine if the system remains in a safe state after the allocation.

3. **Decision**:
    - If the state is safe, the allocation is allowed.
    - If not, the allocation is rolled back and the process must wait.

Advantages:

- Prevents deadlocks **before** they occur.
- Ensures that system remains **safe** and **stable**.
- Efficient for systems where resource usage patterns are predictable.

Limitations:

- Requires each process to declare **maximum resource needs** in advance.
- Not suitable for systems where resource needs are **dynamic** or unpredictable.
- Additional **overhead** due to frequent safety checks.

Applications:

- Operating Systems (for managing CPU, I/O devices, and memory).
- Distributed databases and transaction systems.
- Cloud computing environments with dynamic resource allocation.
- Real-time systems where deadlock is unacceptable.

Example:

Suppose there are 3 resource types (A, B, C) and 5 processes. The system maintains matrices for **maximum**, **allocated**, and **available** resources. Each time a process requests resources, the algorithm checks whether granting the request keeps the system in a safe state. If yes, the request is granted; otherwise, it is deferred.

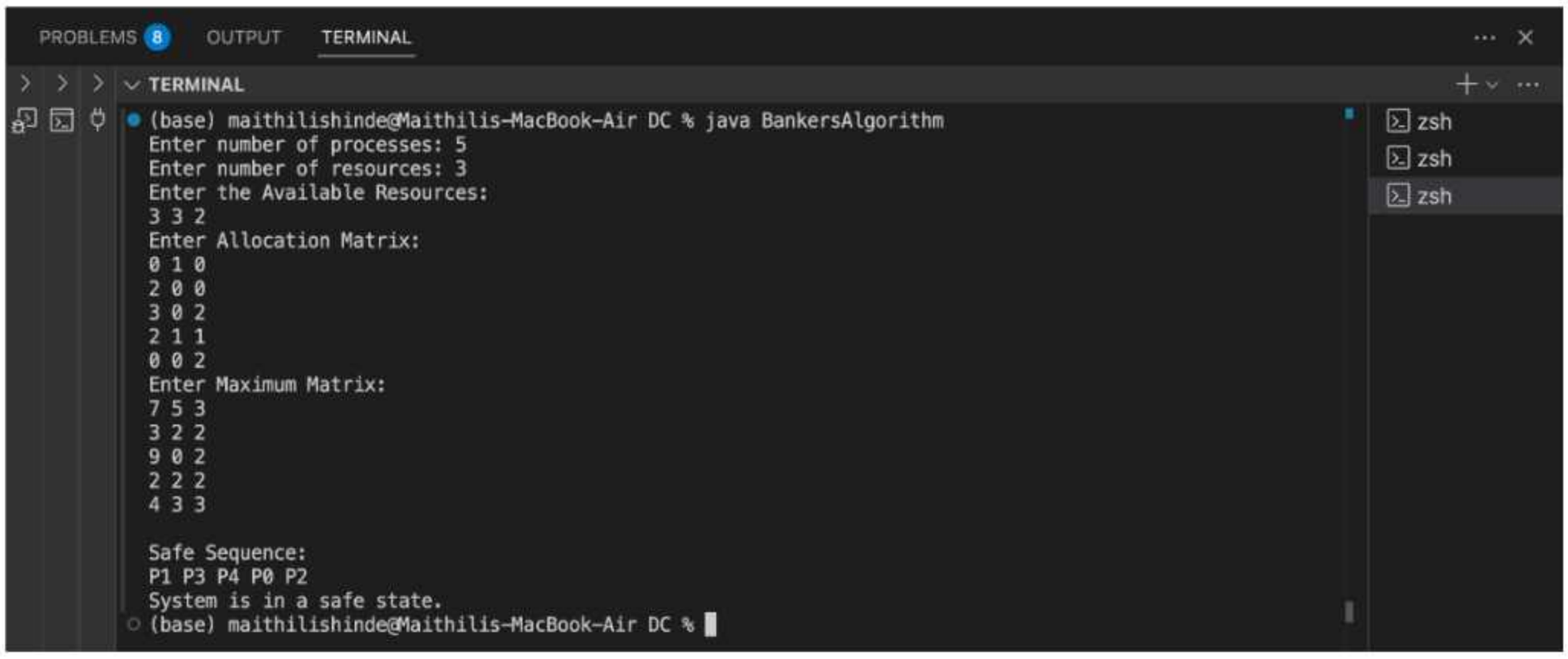
Code & Output:

```
import java.util.Scanner;

public class BankersAlgorithm {
    private int numProcesses, numResources;
    private int[] available;
    private int[][] maximum, allocation, need;

    public BankersAlgorithm(int processes, int resources) {
        numProcesses = processes;
```

```
        numResources = resources;
        available = new int[resources];
        maximum = new int[processes][resources];
        allocation = new int[processes][resources];
        need = new int[processes][resources];
    }

    public void inputResources(Scanner sc) {
        System.out.println("Enter the Available Resources: ");
        for (int i = 0; i < numResources; i++) {
            available[i] = sc.nextInt();
        }
    }

    public void inputAllocation(Scanner sc) {
        System.out.println("Enter Allocation Matrix: ");
        for (int i = 0; i < numProcesses; i++) {
            for (int j = 0; j < numResources; j++) {
                allocation[i][j] = sc.nextInt();
            }
        }
    }

    public void inputMaximum(Scanner sc) {
        System.out.println("Enter Maximum Matrix: ");
        for (int i = 0; i < numProcesses; i++) {
            for (int j = 0; j < numResources; j++) {
                maximum[i][j] = sc.nextInt();
                need[i][j] = maximum[i][j] - allocation[i][j];
            }
        }
    }

    public boolean isSafe() {
        boolean[] finished = new boolean[numProcesses];
        int[] work = available.clone();
        int count = 0;

        System.out.println("\nSafe Sequence: ");
        while (count < numProcesses) {
            boolean found = false;
            for (int i = 0; i < numProcesses; i++) {
                if (!finished[i]) {
                    boolean canAllocate = true;
                    for (int j = 0; j < numResources; j++) {
                        if (need[i][j] > work[j]) {
                            canAllocate = false;
                            break;
```

```
    }
}
if (canAllocate) {
    for (int j = 0; j < numResources; j++) {
        work[j] += allocation[i][j];
    }
    System.out.print("P" + i + " ");
    finished[i] = true;
    found = true;
    count++;
                }
            }
        }
        if (!found) {
            System.out.println("\nSystem is in unsafe state!");
            return false;
        }
    }
    System.out.println("\nSystem is in a safe state.");
    return true;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter number of processes: ");
    int processes = sc.nextInt();
    System.out.print("Enter number of resources: ");
    int resources = sc.nextInt();

    BankersAlgorithm ba = new BankersAlgorithm(processes, resources);
    ba.inputResources(sc);
    ba.inputAllocation(sc);
    ba.inputMaximum(sc);

    ba.isSafe();
    sc.close();
  }
}
```

Output:


PROBLEMS 8
OUTPUT
TERMINAL
TERMINAL
(base) maithilishinde@Maithilis-MacBook-Air DC % java BankersAlgorithm
Enter number of processes: 5
Enter number of resources: 3
Enter the Available Resources:
3 3 2
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Safe Sequence:
P1 P3 P4 P0 P2
System is in a safe state.
(base) maithilishinde@Maithilis-MacBook-Air DC %
zsh
zsh
zsh

EXPERIMENT 7

AIM: Write a program to simulate Load balancing Algorithm

Theory:

In a distributed system, tasks are executed across multiple interconnected computers or nodes. An imbalance in task distribution can lead to some nodes being overloaded while others remain underutilized. **Load balancing** is the technique of dynamically distributing workloads across nodes to ensure no single node is overwhelmed.

The goal of load balancing is to **maximize throughput**, **reduce response time**, and ensure **efficient resource usage**. It also plays a crucial role in fault tolerance and system scalability.

Key Concepts:

- **Static Load Balancing**: The load is distributed in advance based on known parameters.
- **Dynamic Load Balancing**: Load is assigned based on current system conditions.
- **Workload Distribution**: Ensuring all nodes perform roughly equal amounts of work.
- **Node Monitoring**: Tracking load on each node to make informed distribution decisions.

Working Principle:

1. **Task Queue**: The system maintains a queue of tasks that need to be processed.
2. **Node Status Check**: The load balancer regularly checks the current load on each node.
3. **Task Assignment**: Tasks are assigned to the **least-loaded** node to maintain balance.
4. **Update Load**: After assignment, the node's load status is updated.
5. **Completion Tracking**: Once the task completes, the load value is reduced accordingly.

Types of Load Balancing Algorithms:

- **Round Robin**: Tasks are distributed in a circular order, regardless of current load.
- **Least Connections**: The task is assigned to the node with the fewest current tasks.
- **Random Selection**: Nodes are randomly selected for task assignment.
- **Weighted Distribution**: Nodes are assigned tasks based on their processing capacity.
- **Work Stealing**: Idle nodes take over tasks from overloaded ones.

Advantages:

- Improved resource utilization.
- Enhanced system throughput.
- Reduced task response time.
- Avoidance of system crashes due to node overloading.
- Better scalability and fault tolerance.

Limitations:

- Overhead of continuous load monitoring in dynamic algorithms.
- Complexity in implementation of adaptive balancing strategies.
- Inconsistent performance if node capacities vary widely.

Applications:

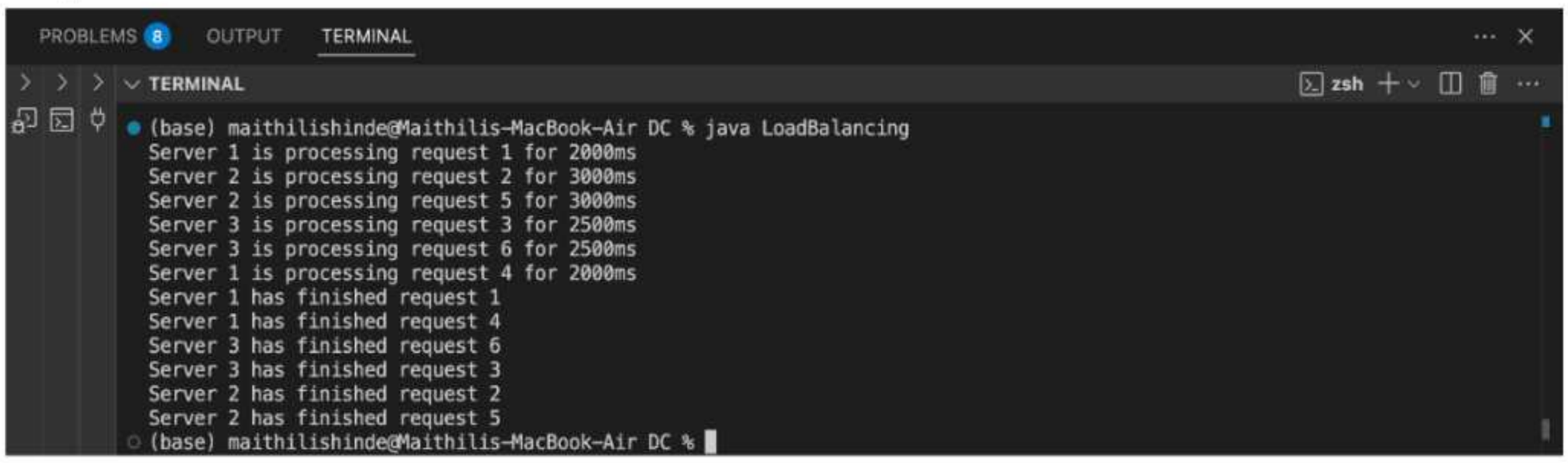
- Cloud and edge computing
- Web servers (like NGINX, HAProxy)
- Distributed file systems
- High-performance computing clusters
- Microservices and container orchestration systems (like Kubernetes)

Code & Output:

```
import java.util.*;

class Server {
    private String name;
    private int processingTime;

    public Server(String name, int processingTime) {
        this.name = name;
        this.processingTime = processingTime;
    }

    public void handleRequest(int requestId) {
        new Thread(() -> {
            System.out.println(name + " is processing request " + requestId + " for " +
processingTime + "ms");
            try {
                Thread.sleep(processingTime); // Simulating processing time
            } catch (InterruptedException e) {
                System.out.println(name + " encountered an error.");
            }
            System.out.println(name + " has finished request " + requestId);
        }).start();
```

```
    }
}

class LoadBalancer {
    private List<Server> servers;
    private int currentIndex = 0;

    public LoadBalancer(List<Server> servers) {
        this.servers = servers;
    }

    public void distributeRequests(int totalRequests) {
        for (int i = 1; i <= totalRequests; i++) {
            Server server = servers.get(currentIndex);
            server.handleRequest(i);
            currentIndex = (currentIndex + 1) % servers.size(); // Round-robin
        }
    }
}

public class LoadBalancing {
    public static void main(String[] args) {
        List<Server> servers = Arrays.asList(
            new Server("Server 1", 2000),
            new Server("Server 2", 3000),
            new Server("Server 3", 2500)
        );

        LoadBalancer lb = new LoadBalancer(servers);
        int totalRequests = 6;
        lb.distributeRequests(totalRequests);
    }
}
```

Output:



Experiment :- 8

Aim :- Distributed shared memory

Theory :-
Distributed shared memory (DSM) is a memory management technique that allows multiple computers in a distributed system to share a logical memory space. This concept provides an abstraction that enables processes on different nodes to access shared data as if it were a local memory, eliminating the need for explicit message passing.

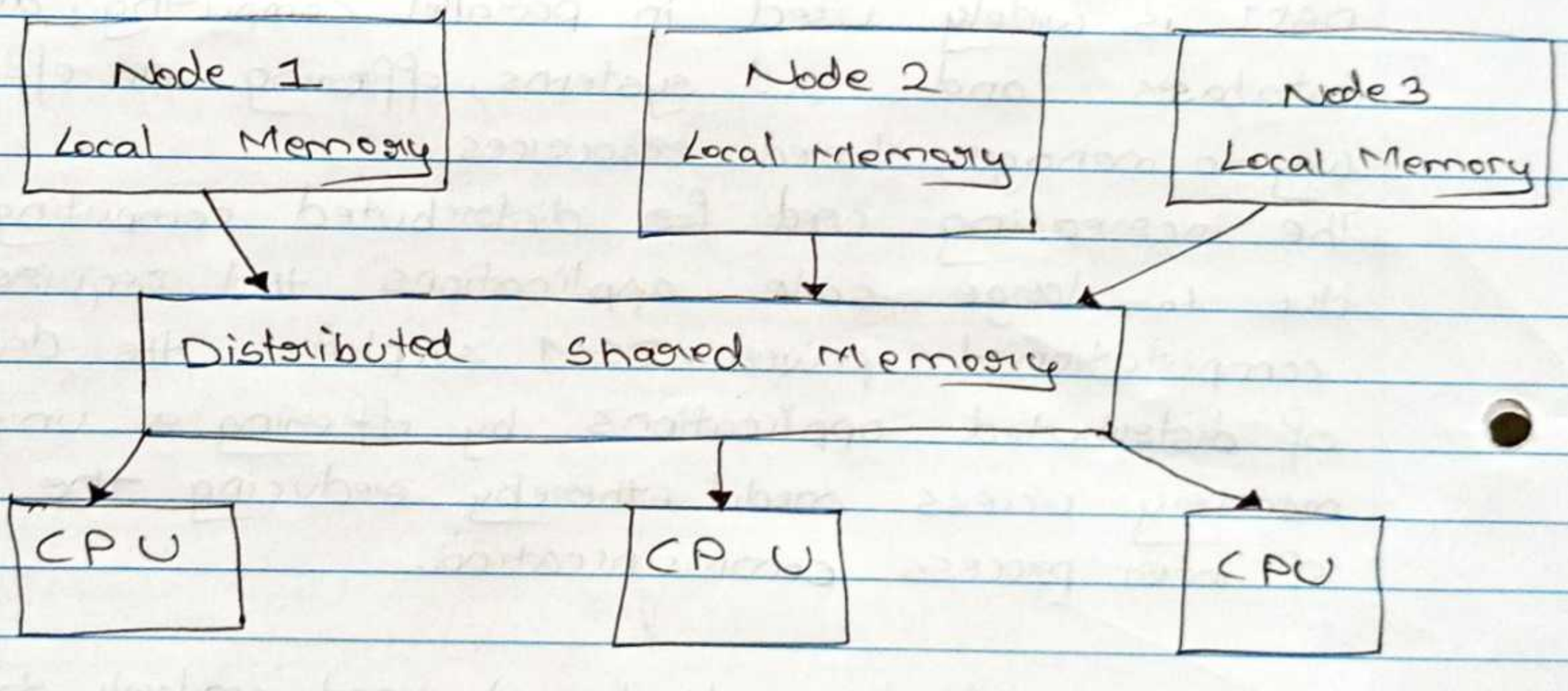
DSM is widely used in parallel computing, distributed databases and cloud systems, offering an efficient way to manage shared resources.
The increasing need for distributed computing arises due to large scale applications that require massive computational power. DSM simplifies the development of distributed applications by offering a unified memory access model, thereby reducing the complexity of inter process communication.

It allows applications to be designed similarly to shared processor systems, making programming more intuitive.

→ Architecture of DSM :-
DSM systems consists of multiple processing nodes, each with its own local memory, connected through

a high speed network. The DSM software layer manages data consistency and synchronisation across these nodes.

The architecture can be categorised into:
1) Hardware based DSM :- Implemented at the hardware level using special memory controllers that enable data sharing among processors.
2) Software based DSM :- Implemented as a layer of software over a network distributed system, which manages data consistency and access control.
3) Hybrid DSM :- A combination of both hardware and software techniques to optimise performance.



This architecture ensures that each node can access shared memory without directly communicating with other nodes.

Memory consistency models in DSM:-
Maintaining consistency in DSM is crucial to ensure

correct execution of distributed computing applications. Different consistency models define how memory updates are propagated.

1) Strict consistency - Every read operation returns the most recent write operation all nodes.
2) Sequential consistency - All nodes observe memory modifications in the same order but execution can be non-deterministic.
3) Causal consistency - Ensures that memory updates that are causally related are seen in the correct sequence.
4) Eventual consistency - Updates propagate gradually and all nodes eventually reach a consistent state. This model is used in distributed databases like amazon, DynamoDB and Apache cassandra.

Implementation techniques in DSM:
DSM systems employ different methods to manage shared memory and improve efficiency.

1) Page-based DSM:- The shared memory is divided into fixed size pages that are dynamically mapped to different nodes. When a node accesses a page that is not available locally it retrieves it from another node.

2) object-based DSM :- Instead of pages, objects are used as the unit of sharing. This method simplifies data access and reduces memory overhead.

3) segment-based DSM :- Uses variable sized

segment instead of fixed pages allowing flexible memory allocation and better performance.

Each method has advantages and trade offs in terms of latency, memory overhead and synchronisation complexity.

→ Advantages of DSM :-

1) Simplified programming - Developers can use shared memory location, abstraction rather than explicit message passing mechanism.

2) Scalability - New nodes can be added dynamically to the system without major architectural changes.

3) Improved resource utilisation - Multiple nodes can efficiently share memory, reducing redundancy.

3) Fault tolerance - DSM systems can replicate memory across nodes, ensuring data availability even in case of node failures.

→ Challenges in DSM :-

1) High communication overhead - Frequent memory accesses from remote nodes can lead to network congestion.

2) Synchronisation issues - Ensuring consistency and avoiding race conditions requires synchronization mechanism which are very much sophesticated.

3) Latency problems - Accessing remote memory is inherently slower than accessing local memory affecting performance.

4) Security concerns - since multiple nodes access the same memory space, data integrity and security must be carefully managed.

→ Comparison of DSM with traditional shared memory.

Traditional Shared Memory	Distributed Shared Memory
1) Single system memory.	1) Multiple distributed nodes.
2) No network involved	2) Network based memory access.
3) Faster access	3) Network latency affects access speed.
4) Limited by hardware.	4) Highly scalable.
5) Faster Synchronisation	5) Requires consistency models.

→ Examples applications of DSM :-

1) Distributed databases -
In distributed database system, multiple users access and modify shared records across different geographical locations. DSM ensures that all updates are propagated correctly preventing inconsistencies.

Examples include Google big table, Apache HBase and amazon aurora.

2) Cloud computing -
Cloud service providers use DSM to offer scalable virtual memory solutions. Applications like kubernetes and Hadoop rely on DSM for efficient distributed resource allocation.

3) Distributed file systems -
Systems like Google file system (GFS) and Hadoop distributed file system (HDFS) use DSM to manage distributed storage efficiently.

4) Parallel scientific computing -
Applications in weather forecasting, molecular simulations and AI training require parallel computation across multiple nodes. DSM enables seamless data sharing, improving computational efficiency.

Pradge

THADOMAL SHAHANI
TSEC
ENGINEERING COLLEGE

Experiment :- 9

Aim:- Distributed File System

Theory :-

A distributed file system that allow multiple users and applications to access files stored across. different networked computers as if they were located on a single machine.

DFS abstracts the complexities of distributed storage providing a unified interface for accessing and managing files seamlessly. DFS is widely used in cloud computing, big data analytics and enterprise storage solutions.

Modern applications require efficient file storage and retrieval mechanisms that support scalability, fault tolerance and data consistency.

DFS enables organisations to manage large datasets efficiently while ensuring high availability and redundancy.

It forms the backbone of large scale distributed systems, cloud storage platforms and content delivery networks.

→ Architecture of DFS :-

A DFS consists of multiple storage nodes that work together to provide a cohesive file system.

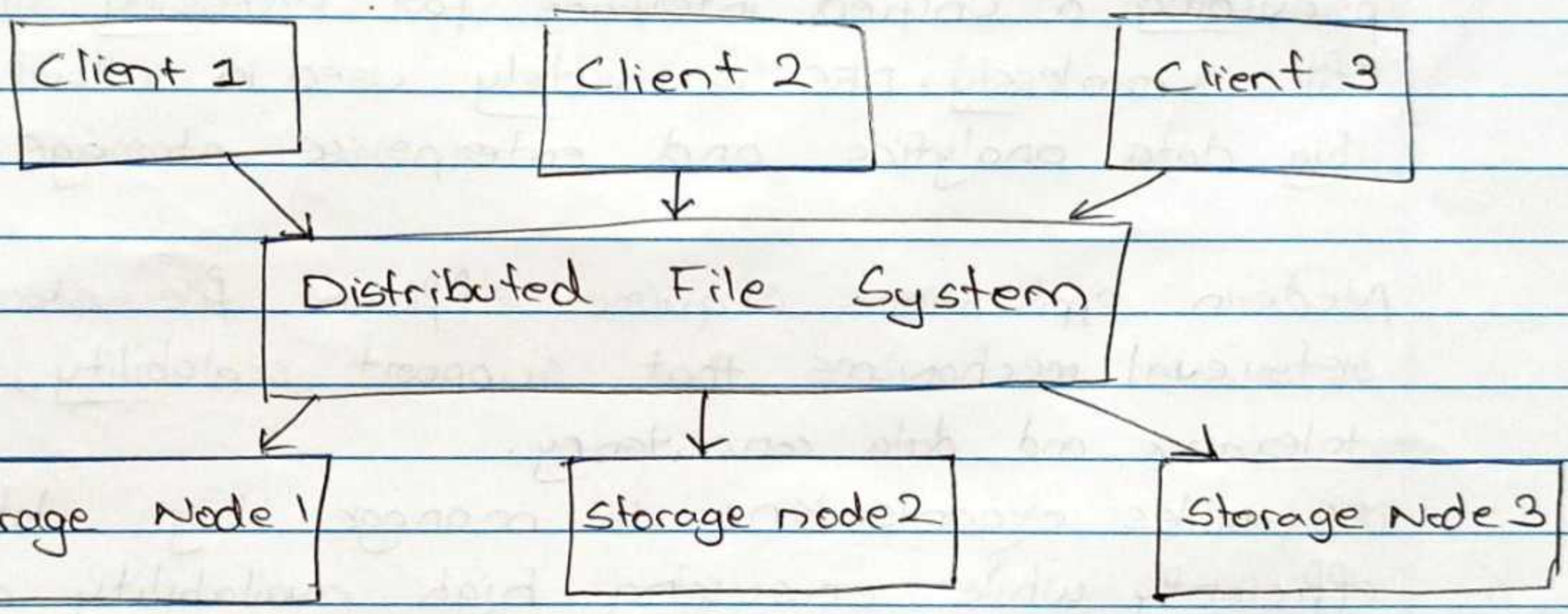
The architecture can be categorised into:

1) Client server model - clients send file requests to the

server, which retrieves or modifies the file stored on distributed nodes.

2) Peer to peer model - Each node in the system can act as both a client and a server, allowing decentralised storage and retrieval.

3) Hybrid model - Combines elements of both client server and peer to peer architectures to optimise performance and reliability.



This architecture ensures that data is distributed across multiple nodes while providing seamless access to clients.

→ Key features of DFS :-

1) Transparency - Users can access files as if they were stored locally, regardless of their physical location.
2) Scalability - DFS can handle large volumes of data and scale horizontally by adding more storage nodes.
3) Fault tolerance - Data replication and redundancy

mechanisms ensure data availability even if some nodes fail.

4) Concurrency control - Multiple users can access and modify files simultaneously while maintaining consistency.

5) Security and access control - Authentication, encryption and permission management ensure the data security of data.

→ Types of DFS:-

1) Location Transparency - Users can access files without knowing their physical location, reducing dependency on specific storage ~~~~ nodes.

2) Replication based DFS-
Data is replicated across multiple nodes to improve reliability and performance. If one node fails, another node provides access to the data.

3) Partitioned DFS -
The file system is divided into partitions with each node responsible for storing and managing a subset of files. This enhances scalability and load balancing.

→ Advantages of DFS-

1) Improved data availability- Replication ensures that file remain accessible even if some nodes go offline.

2) Efficient load balancing- Distributed storage nodes balance files access, requests, reducing bottlenecks.

3) Cost effective- Organisations can levarage ~~~~

commodity hardware to build large scale storage solutions.

4) Support for large datasets - DFS is ideal for handling big data applications that require distributed storage and processing.

5) Remote access - Users can access and collaborate on files from different geographical locations.

→ Challenges in DFS :-

1) Network latency - Accessing remote files introduces delays compared to local file systems.

2) Data consistency - Ensuring consistency in a distributed environment can be complex especially during concurrent modification.

3) Security concerns - Data transmitted over a network is vulnerable to unauthorised access and cyber threats.

4) Metadata Management - Large scale DFS must efficiently handle metadata to track file locations and access permissions.

→ Example applications of DFS :-

1) Cloud storage services - Popular cloud storage platforms like google drive, Dropbox and one drive uses DFS to provide seamless file access and synchronisation across devices.

2) Big data frameworks - Hadoop distributed file system (HDFS) is a key component of big data ecosystems, enabling efficient storage and retrieval of massive datasets.

3) Content delivery networks - CDNs are like

Akamai and cloudflare use DFS to store and distribute web content, ensuring fast and reliable access to users worldwide.

4) Enterprise file systems - Large enterprises use DFS solutions to manage shared file access across multiple office locations, improving collaborations and data management.

→ Comparison of DFS with traditional file system:-

Traditional File System	Distributed File System
1) It consists of centralised storage.	1) It has storage distributed across nodes.
2) Performance is limited to a single machine.	2) Performance is scalable across network.
3) Single point of failure.	3) Redundant data storage.
4) Scalability is limited by hardware.	4) It is horizontally scalable.
5) Access speed is faster. (local)	5) Access speed is dependent on network latency.

With advancements in AI, edge computing and blockchain, DFS is evolving to support decentralised and intelligent data storage solutions.
Future DFS implementations will likely incorporate machine learning for optimised data placement enhanced security through encryption techniques and better integration with hybrid cloud environment.

THADOMAL SHAHANI TSEC ENGINEERING COLLEGE

Experiment :- 10

Aim :- ~~Caste~~ Case study on ~~CORBA~~ CORBA and android stack

Theory :-

Common object request broker architecture (CORBA) and android stack are two significant technologies in the world of distributed computing and mobile application development. CORBA is a middleware standard that enables communication between objects in a distributed network, while the Android stack represents the software architecture that powers an droid devices.

CORBA is a standard ~~devt~~ defined by the object management group (OMG) that allows software components to communicate regardless of the programming language or platform they are running on. It provides a common interface for distributed object interaction using the object request broker (ORB) which acts as a mediator between client requests and server responses. CORBA is widely used in enterprise applications, financial systems and telecommunication.

→ Architecture of CORBA :-

CORBA consists of key components including the ORB interface definition language (IDL) and the CORBA services. The ORB facilitates communication between

clients and servers, while the IDL defines object interfaces independent of programming languages. The POA manages object lifecycles and CORBA services to provide essential functions like naming, security and transaction management.

→ Advantages of CORBA:-
One of the primary benefits of CORBA is its interoperability across different platforms and languages. It supports a wide range of languages including C++, Java and Python making it suitable for heterogeneous environments. CORBA also offers robust security features, high scalability and efficient communication mechanisms, making it an ideal choice for large scale distributed system.

→ Challenges in using CORBA:-
Despite its advantages, CORBA faces challenges such as complexity, overhead and performance issues. Setting up and maintaining a CORBA based system requires significant effort and its performance may not be optimal for modern web based applications. Additionally the rise of newer ~~technology~~ technologies like web services and RESTful APIs has reduced the adoption for CORBA in recent years.

→ Android Stack:-
An android stack is the layered ~~stack~~ software ~~architecture~~ that powers android operating systems. It consists of four main layers:

1) The Kernel layer
2) Hardware abstraction layer
3) Android runtime (ART)
4) Application framework.

These layers ~~provide~~ work together to provide a secure, efficient and flexible environment for running android applications.

→ Components of Android Stack :-
The linux kernel serves as foundation of the android stack, handling low level operations like memory management, process scheduling and hardware communication. The HAL provides standard interfaces for interacting with device hardware, allowing manufacturers to implement custom drivers.

The Android Runtime (ART) is responsible for executing applications using Just-in-time (JIT) and Ahead-of-time (AOT) compilation techniques. The application framework provides high level APIs for developing Android applications, including components like activities, services and content provision.

→ Advantages of Android Stack :-
The android stack offers several benefits, including open source development, extensive hardware support and a robust application ecosystem. Its modular architecture allows developers to build highly customisable applications, while its security features

ensure data protection and application sandboxing. Additionally the widespread adoption of Android devices has created a thriving developer community contributing to continuous innovation in the ecosystem.

→ Challenges in the Android Stack :-
Despite its strengths, the android stack has certain limitations such as fragmentation, security vulnerabilities and performance optimisation challenges.
The diversity of Android devices and manufacturers lead to inconsistencies in software update and application compatibility Additionally ensuring optimal performance across different hardware configurations requires significant effort from developers.

→ Real world applications of CORBA :-
CORBA is extensively used in mission critical applications where reliability and scalability are paramount. Ex include-
1) Banking and financial services - CORBA enables secure transactions and integration of legacy systems.
2) Telecommunications - Used for managing network operations and services across distributed environments.
3) Healthcare Systems - Facilitates interoperability between different medical applications and data exchange standards.

→ Real world applications of Android Stack:-

1) Mobile applications - Android is the dominant OS for smartphones and tablets, supporting millions of apps on Google play.
2) Embedded Systems - Many IOT and smart devices use Android due to its flexibility and customisation options.
3) Automotive Industry - Android automotive is used in modern infortainment systems, offering seamless integration with smartphones.
4) Wearable technologies - smartwatches, fitness trackers and other wearables run on android based platforms.

→ CORBA	Android Stack
1) It is for middleware for distributed systems.	1) It is for mobile applications development framework.
2) It's architecture is object oriented and uses ORB and IDL.	2) It has layered architecture with linux Kernel.
3) It's primarily used for enterprise applications financial services.	3) It's primarily used for mobile, IOT and embedded applications.
4) Performance can be high but suffers from overhead.	4) Performance is optimised for mobile devices.

DC

THADOMAL SHAHANI TSEC ENGINEERING COLLEGE

Assignment :- 1

Q1 Explain issues in designing distributed systems

Ans 1 A distributed system is a collection of autonomous computer systems that are physically seperated but are connected by a centralised computer system, that is equipped with distributed system software.

Design issues of distributed systems :-

1) Scalability - As the number of users or requests increases the system must scale accordingly without performance degeradation.

There are two main approaches of scalability that ~~can~~ are horizontal and vertical scaling. Horizontal scaling involves adding more nodes to the system to distribute the load, while vertical scaling involves increasing capacity of existing nodes (eg. adding CPUs, memory or storage).

Load balancing is also a key factor, as an uneven distribution of requests can lead to bottlenecks and increased latency.

2) Fault . tolerance and availability - A major advantage of distributed systems is their ability to remain operational even when individual nodes fail.

However achieving fault tolerance requires replication and redundancy. Replication ensures that multiple copies of data exist across different nodes that if one node fails the another can take over without data loss.

This introduces the challenge of keeping replicas synchronised and consistent, especially when multiple copies of data

exists and multiple updates are occuring simultaneously. Network failures are other challenges as nodes may lose connections with each other temporarily.

3) Consistency -
Ensuring data consistency across nodes is one of the most challenging aspects of distributed system design due to the cap (Consistency, Availability, Partition Tolerance) theorem.
Consistency is often used in distributed systems where nodes may hold different data temporarily but they will converge eventually to the state. Strong consistency guarantees, where every node reflects the latest state immediately, are harder to achieve and may require global locks or complex synchronisation in protocols which can reduce system performance.

4) Concurrency -
Distributed systems often handle multiple requests simultaneously which introduces concurrency issues such as race conditions and deadlocks. A race condition occur when two or more nodes are waiting for each other to release resources causing the system to halt.
Conflict resolution is also a key challenge when different nodes update the same data concurrent.

THADOMAL SHAHANI TSEC ENGINEERING COLLEGE

Q2 Differentiate between NOS, DOS and Middleware in the design of a distributed system.

Ans 2

Network operating System (NOS)	Distributed operating System (DOS)	Middleware
1) An operating system that enables computers connected on a network to communicate and share resources.	1) An operating system that manages a group of computers and presents them as a single system to user.	1) A software layer that provides common services and communication between applications in distributed environment.
2) Facilitates resource sharing and communication between independent computers.	2) Manages distributed resources and processes as they are part of a single system.	2) Acts as a bridge between operating system and app providing consistent platform.
3) It provides mechanisms for file sharing and communication between independent computers.	3) Handles distributed resources transparently making them appear local to the user.	3) Provides abstraction for resource sharing through API's and communication protocols.
4) Limited transparency. The user is aware of the network and remote resources.	4) High transparency. The user perceives the system as a single entity.	4) Partial transparency hides complexity of communication and coordination.
5) Fault tolerance is low as failure of one node can affect the whole system.	5) Fault tolerance is high as OS system can continue functioning even if some nodes fail.	5) Medium can handle some nodes failure but depends on underlying OS and network.

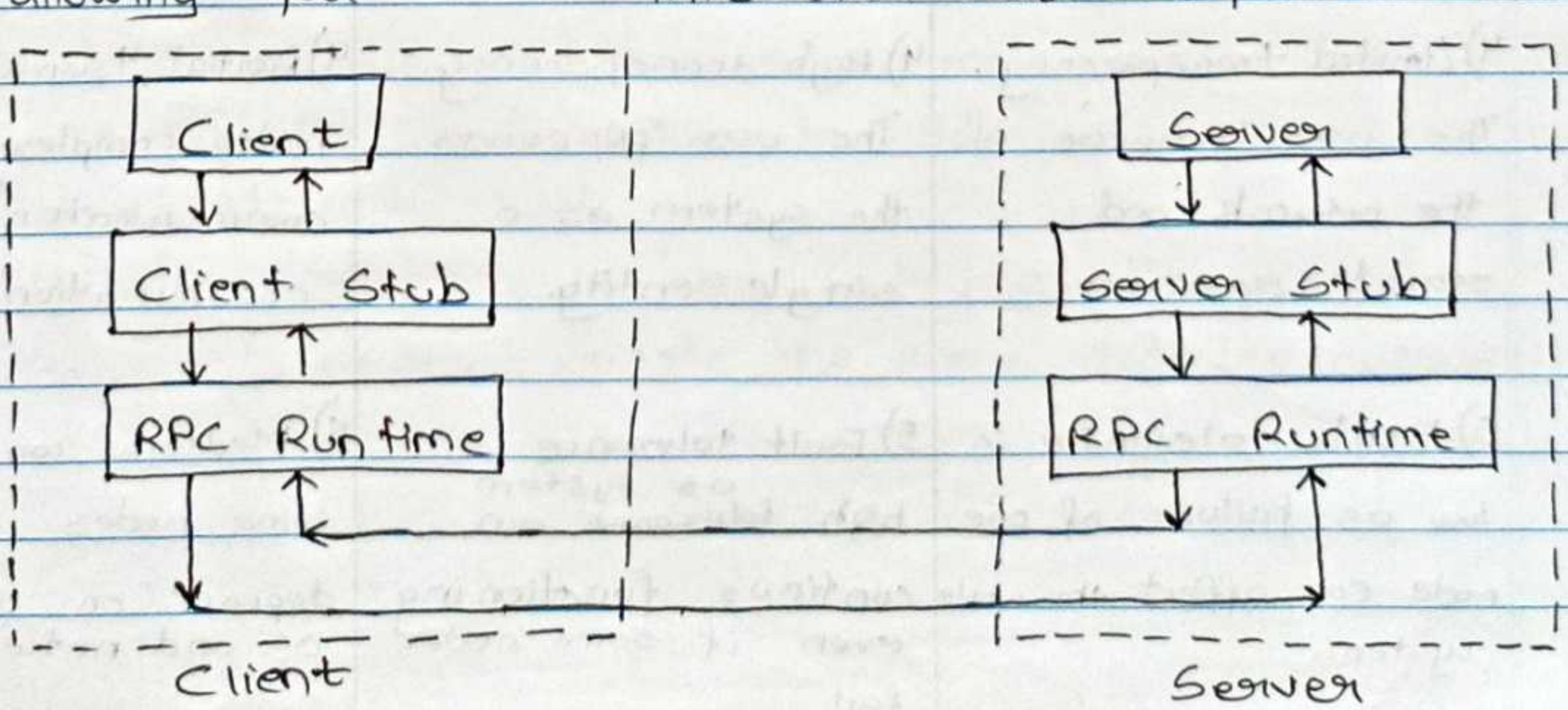
6) Independent process management for each node.	6) Centralized and coordinated process management across nodes.	6) Provides mechanisms for process coordination and communication
7) ex :- windows server, linux Samba, Novell Netware.	7) ex :- Amoeba, Sprite, plang.	7) ex :- CORBA, RPC, Java RMI

Q3 Define Remote Procedure Call. Describe the working of RPC in detail.

Ans 3 A Remote Procedure Call (RPC) is a protocol in distributed system that allows a client to execute functions on a remote server as if they were local RPC simplifies the network communication by abstracting the complexities, making it easier to develop and integrate distributed applications efficiently.

RPC enables a client to invoke methods on server residing in a different address space (often on a different machine) as if they were local procedures.

The client and server communicate over a network allowing for remote interaction and computation.



The working of RPC involves several key steps which ensure that the procedure call made on one machine is executed on remote machine and the result is returned to the caller.

Client Stub generation :-

When a program makes an RPC call, it first generates a client stub. The client stub is a local representation of the remote procedure, ~~call~~ hiding the details of network communication. The client stub serializes (marshals) the procedure call, including the procedure name, input parameters and other necessary information.

Marshalling :-

Marshalling refers to the process of converting the procedure name into format suitable for a transmission over the network. This usually involves converting data structures into byte stream. The client stub takes care of marshalling data, ensuring that the format is consistent with the format expected by the remote server.

Transport and communication :-

After marshalling the client stubs sends the request over the network using a communication protocol (eg. TCP/IP). The network layer handles packet transmission, error checking and retransmission if needed. The client stub remains blocked (waiting) until a response is received from the server.

Server stub and Unmarshalling :-

On the server side, a server stub receives the request and unmarshals (deserializes) the data, converting the byte stream back into the procedure name and input parameters. The server stub then forwards the request

to the actual procedure on the server.

Procedure execution :-

The server executes the requested procedure using the provided parameters. After execution the procedure generates a result (or an error) which needs to be sent back to the client.

Marshalling of the result :-

The server stub marshalls the result (or error message) into a byte stream, ~~which~~ similar to the request marshalling. This ensures that the results can be transmitted back to the client over the network.

Transport and communication :-

The network layer on the server sends the result back to the client over the established connection. If there is a network failure and timeout, the request may need to be resent.

Unmarshalling of result :-

The client stub receives the results and unmarshal it back into a format that can be understood by the client program. This involves converting the byte stream into ~~the~~ structured data.

Return of Result :-

The client stub finally returns the result to the calling program, making it appear as though the procedure was executed locally. If an error occured during execution or transmission, the client stub will handle it based on the defined error handling mechanism.

Q4 Write a note on group communication.

Ans 4 Group discussion in distributed computing refers to the process where multiple nodes or processes in distributed system communicate and collaborate to reach a common decision, solve a problem or coordinate their actions. Since distributed system consists of independent node that may operate asynchronously and encounter network failures, effective group discussion mechanisms are essential for maintaining consistency and ensuring coordinated behavior across the system.

The primary purpose of group discussion ~~is in distribution~~ distributed computing is to enable nodes to work together and make consistent decisions despite challenges such as network latency, node failures and message loss. Distributed system requires agreement on system state, task allocation ~~and~~, resource sharing and failure recovery which necessitate structured and reliable communication protocols.

Group discussion is distributed computing relies on several key mechanisms to ensure nodes can communicate and make content decisions.

One important mechanism is broadcast and multicast communication, where messages either ~~help~~ sent to all nodes (broadcast) or a specific group of nodes (multicast). This allows all participating nodes to receive same information, enabling synchronized decision making.

Another critical mechanism is consensus algorithms which ensure that the nodes agree on a single value or state even when some nodes fail.

Paxos and Raft are widely used consensus algorithms

that rely on the voting process to achieve consistency. Two phase ~~out~~ commit (2PC) and three phase commit (3PC) are also used for transaction consistency where nodes vote to either commit or abort a transaction.

THADOMAL SHAHANI TSEC ENGINEERING COLLEGE

DC

Assignment :- 2

Q1 What is the need for code migration ? Explain the role of process to resource and resource to machine binding in code migration.

Ans 1 Code migration refers to process of transferring code execution from one machine to another in distributed system.

The need for code migration arises from the requirement to improve system performance, balance the load, enhance ~~the~~ fault tolerance, and reduce network latency.

In distributed systems different machines have different processing capabilities, storage capacities and network connections.

Migrating code to a machine with higher processing power or better network connectivity can significantly enhance the system's overall performance.

Code migration is also useful in scenarios where certain resources and data are available only on specific nodes and moving the code closer to those resources reduces the cost of data transmission and processing time.

Additionally it helps in ~~moving~~ improving system scalability by distributing the workload evenly across multiple nodes.

Code migration also supports fault tolerance by allowing tasks to be transferred to a different machine in case of hardware of network failures.

~~Process to resource~~

Process to resource and resource to process machine binding in code migration.

In code migration, the relationship between process and resource

and between resources and machines, plays a critical role in determining the effectiveness of migration.

Process of resource binding :-
In distributed systems, processes need to access various resources such as files, databases, network connections and memory. Process to resource binding refers to how tightly a process is linked to a specific resource.
If a process is tightly bound to a resource located on a particular machine, migrating the process to another machine may require establishing a new connection to the resource or transferring the resource itself.
There are three types of process to resource building-
i) By identifier - When a process is bound ~~of~~ a to a specific resource by an identifier, migrating the process becomes complex because the identifier is often machine specific.
ii) By value - When a process is bound to ~~to~~ a resource by value, migration is easier since the resources value can be transferred along with the process.
iii) By type - When a process is bound to a type of resource (such as accessing a database server) the system can redirect the process to a different instance of the resource after migration.

Resource of machine binding:-
Resource to machine binding refers to how resources are associated with specific machines in the distributed system.
There are three types of resource to machine learning-
i) Fixed binding - A resource is permanently attached to a specific machine. Migration is impossible unless the

process adopts to an alternative resource.

ii) Fastened binding - A resource is initially attached to machines but can be moved if needed. Code migration is possible but it may require re-establishing the resource connection.

iii) Loose binding - A resource is not tied to any specific machine and can be accessed from any node. This allows flexible code migration since the process can easily reconnect to the resource from a new machine.

Q2 List desirable features of distributed file system. How are modifications propagated in file caching schemes?

Ans 2 A distributed file system (DFS) allows files to be stored and accessed across multiple networked nodes while providing replication transparency.

i) Scalability is essential for handling a growing number of files, users and nodes without performance issues. The system should dynamically adjust to increased load by adding new nodes without requiring reconfiguration. High performance is important with low latency for file access and efficient handling of concurrent operations.

ii) Consistency ensures that file modifications are reflected across all replicas and users. Strong consistency models ensure immediate updates, while eventually consistency allows some delay in propagation. Fault tolerance is critical, where the system should recover from node or network failures automatically using replication and redundancy.

iii) Security should be maintained through encryption, authentication and access control to prevent unauthorised access and data corruption. Concurrency is essential to allow multiply to read and

modify files simultaneously without conflicts, which can be managed using file locking and version control.

iv) Replication increases availability by storing multiple copies of files across different nodes. The system should select the nearest or least busy replica for access. Heterogeneity ensures that the DFS works across different operating systems and hardware. Finally data integrity should be preserved using checksums and error correcting codes to detect and repair corrupted files.

How modifications are propagated in file caching.
In write through caching, file modifications are immediately written to the central server and all replicas. This approach ensures strong consistency but increases write latency because the client must wait for confirmation from the server. It is suitable for application where accuracy and consistency are more important than speed.

In write-back caching, file modifications are made locally and propagated to the server and replicas at regular intervals or when the file is closed. This reduces write latency and improves performance but introduces a risk of data loss if the system crashes before the changes are synchronised.

In lease-based caching, the server grants a lease to the client, allowing local modifications for a fixed time. When a lease expires, the client synchronises

the changes with the server. This approach balances consistency and performance by limiting the time window for inconsistency. It reduces the unnecessary updates while ensuring eventual consistency. The updates are pushed when the lease expires.

A
Pradyn